

media_validation_tools_20260621

Media Validation Tools — Research Report

Field	Value
Purpose	Catalogue of automated media validation tools for the Helix OTA §11.4.163 pipeline
Revision	1
Last modified	2026-06-21T00:00:00Z
Scope	OCR/Vision, video analysis, audio analysis, asciicast, full-stack frameworks
Cross-reference	§11.4.107 (liveness/freeze/correctness), §11.4.117 (CV/OCR fallback), §11.4.136 (real-content), §11.4.137 (subtitle), §11.4.158 (intensive recording), §11.4.159 (vision validation)

Table of Contents

1. [OCR / Vision Tools](#)
2. [Video Analysis Tools](#)
3. [Audio Analysis Tools](#)
4. [Asciinema Cast Analysis](#)
5. [Automated Content Verification Frameworks](#)
6. [Comparison Matrix](#)
7. [Recommended Pipeline Architecture](#)

1. OCR / Vision Tools

1.1 Tesseract OCR

Purpose: Open-source OCR engine for printed text recognition. Mature, battle-tested, community-supported.

How it works: Traditional pipeline — image preprocessing (binarization, deskew, noise reduction) → character segmentation → LSTM neural recognition (since v4) → language-model post-correction. Supports 100+ languages via trained language data files.

Accuracy (v5.3.x): | Scenario | Accuracy | |—|—| | Clean printed English, >=300 DPI | ~97%+ | | Handwritten text | ~89%+ | | Multi-column / complex layout | Lower — needs preprocessing | | Chinese with preprocessing | ~89-95% |

Speed: Fast on CPU. No GPU needed. Suitable for high-throughput batch pipelines.

Installation:

macOS

```
brew install tesseract tesseract-lang
```

Ubuntu/Debian

```
sudo apt install tesseract-ocr tesseract-ocr-eng tesseract-ocr-chi-sim
```

Python wrapper

```
pip install pytesseract
```

Bash pipeline example:

Basic OCR on an image file

```
tesseract input.png output.txt -l eng --psm 6
```

With ImageMagick preprocessing

```
convert input.png -colorspace Gray -threshold 50% - | tesseract  
stdin stdout -l eng --psm 6
```

Batch process frames from video

```
for f in ./frames/*.png; do  
    tesseract "$f" "ocr/${basename "$f"}.png" -l eng --psm 6 2>/dev/null  
done
```

Key flags: | Flag | Purpose | |—|—| | --psm N | Page segmentation mode (3=auto, 6=uniform block, 11=single text line) | | --oem N | OCR engine mode (1=LSTM only, 3=LSTM+Legacy hybrid) | | -l LANG | Language(s), e.g. eng, eng+deu | | -c tessedit_char_whitelist=... | Restrict output character set |

Integration into bash/Go pipelines: Straightforward. Tesseract reads stdin and writes stdout/stderr. Wrap in `tesseract stdin stdout` for pipe-based pipelines. In Go, use `os/exec` to call tesseract CLI, or use `gosseract` Go binding (`cgo`, wraps `libtesseract`).

Local / offline: YES. Fully local. No API key.

License: Apache 2.0.

Verification for §11.4.159: OCR output can be grepped for expected patterns. Use `grep -c 'expected text'` on tesseract output for PASS/FAIL.

1.2 EasyOCR

Purpose: Pure-Python deep-learning OCR (80+ languages), easy to set up.

How it works: Deep-learning-based (CRAFT text detector + CRNN recognizer). Built on PyTorch.

Accuracy (April 2026 benchmarks): | Scenario | F1 Score | | Printed text | 94.8% | | Tables | 82.1% | | Handwriting | 76.5% | | **Overall** | **89.2%** |

Speed: ~55 pages/min on RTX 3090, ~92 pages/min on RTX 5090. VRAM ~3.1 GB.

Installation:

```
pip install easyocr
```

Python API (bash-pipeline friendly):

```
# ocr_frame.py - callable from bash
import sys, json
import easyocr

reader = easyocr.Reader(['en'], gpu=True)
results = reader.readtext(sys.argv[1])

# Output JSON for bash processing
print(json.dumps([
    {'bbox': r[0],
     'text': r[1],
     'confidence': float(r[2])
    } for r in results]))
```

Bash pipeline:

```
python ocr_frame.py screenshot.png | jq '.[] | select(.confidence > 0.5) | .text'
```

Local / offline: YES. Models downloaded on first use.

License: Apache 2.0.

Pros: Easiest pure-Python setup. Consumes less VRAM. Good for prototyping.

Cons: Lower accuracy ceiling than Surya or PaddleOCR. Not recommended for sustained production OCR.

1.3 PaddleOCR v4

Purpose: High-speed production OCR pipeline with table recognition. Best for CJK.

How it works: PP-OCRv4 architecture — text detection → direction classifier → recognition. Built on PaddlePaddle framework.

Accuracy (April 2026 benchmarks): | Scenario | F1 Score | |—|—| | Printed text | 97.1% | | Tables | 89.8% | | Handwriting | 82.5% | | **Overall** | **93.5%** |

Speed: FASTEST open-source OCR — ~85 pages/min on RTX 3090, ~140 pages/min on RTX 5090. VRAM ~2.8 GB. ~1.9x faster than Surya.

Installation:

```
pip install paddlepaddle paddleocr
```

Python API:

```
from paddleocr import PaddleOCR
import sys, json

ocr = PaddleOCR(use_angle_cls=True, lang='en')
result = ocr.ocr(sys.argv[1], cls=True)

# Output JSON
lines = []
for r in result[0]:
    lines.append({
        'bbox': r[0],
        'text': r[1][0],
        'confidence': float(r[1][1])
    })
print(json.dumps(lines))
```

Table recognition (built-in):

```
from paddleocr import PPStructure
engine = PPStructure(show_log=True)
result = engine('invoice.jpg') # returns HTML table + OCR
```

Bash pipeline:

```
python paddle_ocr.py document.png | jq
    '.[] | select(.confidence > 0.8) | .text'
```

Local / offline: YES. Models downloaded on first use.

License: Apache 2.0.

Best for: High-volume batch processing, CJK documents, table extraction, production pipelines.

1.4 Surya OCR

Purpose: State-of-the-art open-source document OCR (90+ languages) with layout analysis.

How it works: 650M-parameter Vision-Language Model (VLM). Transformer-based. Detects text lines, layout elements (table, image, header, footer, etc.), reading order, and table structure.

Accuracy (April 2026 benchmarks — BEST among open-source): | Scenario | F1 Score |
| Printed text | 97.5% | | Tables | 91.2% | | Handwriting | 85.3% | |
Overall | 94.8% |

Real-world 1960s scanned documents: - Typed: 98.48% - Handwritten: 87.16% - Overall: 97.41%

Speed: ~42 pages/min on RTX 3090, ~72 pages/min on RTX 5090. VRAM ~4.2 GB. ~5 pages/second on RTX 5090 with appropriate batch sizes.

Installation:

```
pip install surya-ocr
```

First-run downloads ~1.3 GB of model weights automatically

CLI usage:

OCR with bounding boxes (default)

```
surya_ocr DATA_PATH --images --output_dir ./results
```

Layout analysis

```
surya_layout DATA_PATH --images
```

Table recognition

```
surya_table DATA_PATH --images
```

Reading order detection

```
surya_order DATA_PATH
```

Python API:

```
from PIL import Image
from surya.recognition import RecognitionPredictor
from surya.detection import DetectionPredictor
from surya.ocr import run_ocr
import sys, json

image = Image.open(sys.argv[1]).convert("RGB")
recognition = RecognitionPredictor()
detection = DetectionPredictor()

predictions = run_ocr([image], [image], detection, recognition)
# Outputs text_lines with bboxes, confidence, text
```

Bash pipeline:

```
surya_ocr screenshot.png --images --output_dir ./ocr_output 2>/dev/null
# Read from ocr_output/results.json
cat ocr_output/results.json | jq '.[].text_lines[] |
    select(.confidence > 0.5) | .text'
```

Layout analysis output labels: Caption, Footnote, Formula, List-item, Page-footer, Page-header, Picture, Figure, Section-header, Table, Form, Table-of-contents, Handwriting, Text, Text-inline-math.

Local / offline: YES. Models cached locally after first download.

License: GPL 3.0 (revenue-based exception: free for <\$5M revenue or <\$5M VC funding).

Best for: Complex layouts, maximum accuracy, document OCR, reading order, table recognition. Handwriting accuracy is far ahead of alternatives.

2. Video Analysis Tools

2.1 ffmpeg / ffprobe

Purpose: The foundational Swiss Army knife for video/audio analysis. Every other tool depends on ffmpeg.

How it works: Multi-format media library. ffprobe extracts metadata; ffmpeg transforms streams and applies filters.

Installation:

```
# macOS
brew install ffmpeg
```

```
# Ubuntu/Debian
sudo apt install ffmpeg
```

2.1.1 Codec Detection (H.264 / AV1 / HEVC)

```
# Get video codec name
ffprobe -v error -select_streams v:0 \
    -show_entries stream=codec_name \
    -of default=noprint_wrappers=1:nokey=1 input.mp4
# Output: h264 (or hevc, vp9, av1...)
```

```
# Get ALL stream details as JSON
ffprobe -v quiet -print_format json -show_streams input.mp4
```

```
# Get codec + resolution + bitrate
ffprobe -v error -select_streams v:0 \
```

```

-show_entries
    stream=codec_name,width,height,bit_rate,r_frame_rate \
-of default=noprint_wrappers=1:nokey=1 input.mp4

```

Bash conditional for codec check:

```

#!/bin/bash
codec=$(ffprobe -v error -select_streams v:0 \
-show_entries stream=codec_name \
-of default=noprint_wrappers=1:nokey=1 "$1")

if [ "$codec" = "h264" ]; then
    echo "PASS: Video is H.264"
elif [ "$codec" = "hevc" ]; then
    echo "FAIL: Expected H.264, got HEVC"
elif [ "$codec" = "av1" ]; then
    echo "FAIL: Expected H.264, got AV1"
else
    echo "FAIL: Unknown codec: $codec"
fi

```

2.1.2 Frame Extraction

```

# Extract one frame per second
ffmpeg -i input.mp4 -vf "fps=1" frames/out_%04d.png

# Extract every Nth frame
ffmpeg -i input.mp4 -vf "select=not(mod(n\,30))" frames/
    out_%04d.png

# Extract a single frame at 10 seconds
ffmpeg -ss 10 -i input.mp4 -vframes 1 frame.png

# Extract frames only during a specific segment
ffmpeg -i input.mp4 -ss 00:01:00 -to 00:02:00 -vf "fps=1"
    segment_frames/%04d.png

```

2.1.3 Freeze Detection (freezedetect filter)

Purpose: Detect frozen/unmoving frames in a video — the §11.4.107 liveness oracle.

```

# Basic freeze detection (2-second minimum freeze, -60dB noise
    tolerance)
ffmpeg -i input.mp4 -vf "freezedetect=n=-60dB:d=2" -an -f null -
    2>&1

# With metadata output to file
ffmpeg -i input.mp4 \
    -vf "freezedetect=n=0.003,metadata=mode=print:file=freeze_log.txt" \
    -map 0:v:0 -f null -

# Parse freeze start/end/duration

```

```
ffmpeg -i "$1" -vf "freezedetect=n=-60dB:d=2,metadata=mode=print"
\
-an -f null - 2>&1 \
| grep -oP 'lavfi\.(freezedetect\.(freeze_start|freeze_end|
freeze_duration)=\K[0-9.]+'
```

Parameters: - n: noise tolerance in dB (default -60dB). Lower = more sensitive. - d: minimum freeze duration in seconds (default 2).

Bash PASS/FAIL pipeline:

```
#!/bin/bash
# Returns PASS if no freeze >= 2 seconds found, FAIL otherwise
input="$1"
if ffmpeg -i "$input" -vf "freezedetect=n=-60dB:d=2" -an -f null
- 2>&1 \
| grep -q "freeze_duration"; then
echo "FAIL: Freeze detected in $input"
exit 1
else
echo "PASS: No freeze detected in $input"
fi
```

2.1.4 Black Frame Detection (blackdetect filter)

```
# Detect near-black video intervals
ffmpeg -i input.mp4 -vf "blackdetect=d=1:pix_th=0.1" -an -f null
- 2>&1

# Parse black_start/black_end
ffmpeg -i input.mp4 -vf "blackdetect=d=1" -an -f null - 2>&1 \
| grep -oP '(black_start|black_end|black_duration):\K[0-9.]+'
```

2.1.5 Scene Detection / Splitting

```
# Detect scene changes using ffmpeg's select filter
ffmpeg -i input.mp4 -vf "select='gt(scene,0.3)',showinfo" -f null
- 2>&1 \
| grep -oP 'pts_time:\K[0-9.]+'
```

2.1.6 Video Quality Metrics

```
# SSIM comparison between two videos
ffmpeg -i reference.mp4 -i distorted.mp4 \
-filter_complex "ssim" -f null -

# PSNR comparison
ffmpeg -i reference.mp4 -i distorted.mp4 \
-filter_complex "psnr" -f null -

# Histogram / waveform per channel
ffmpeg -i input.mp4 -vf "histogram" histogram.mp4
```

2.2 PySceneDetect

Purpose: Python/OpenCV library for scene cut/transition detection and video splitting.

How it works: Content-aware scene detection (histogram-based, threshold-based, adaptive). Splits video at scene boundaries.

Installation:

```
pip install scenedetect --upgrade
```

CLI usage:

```
# Basic scene detection + split
scenedetect -i video.mp4 split-video

# Extract keyframes from each scene
scenedetect -i video.mp4 save-images

# Custom threshold
scenedetect -i video.mp4 detect-content --threshold 27 split-video

# Time range
scenedetect -i video.mp4 time -s 10 -e 60 detect-content split-
video
```

Python API:

```
from scenedetect import detect, ContentDetector,
    split_video_ffmpeg
import json

scene_list = detect('video.mp4', ContentDetector())
# scene_list is a list of (start_time, end_time) tuples

# Export for bash processing
with open('scenes.json', 'w') as f:
    json.dump([
        {
            'start': s[0].get_seconds(),
            'end': s[1].get_seconds(),
            'duration': s[1].get_seconds() - s[0].get_seconds()
        } for s in scene_list], f)
```

Bash pipeline:

```
scenedetect -i recording.mp4 detect-content --threshold 27 split-
video 2>/dev/null
```

Integration into media validation: Use before OCR — OCR only the I-frame (keyframe) of each detected scene, reducing OCR work and ensuring the OCR'd frame is representative of that scene segment.

Local / offline: YES.

License: BSD 3-Clause.

2.3 OpenCV (cv2)

Purpose: Computer vision library for frame-level analysis — motion detection, frame differencing, optical flow, content verification.

Installation:

```
pip install opencv-python numpy scikit-image
```

2.3.1 Frame Differencing / Motion Detection

```
#!/usr/bin/env python3
"""motion_check.py - Detect if video has advancing (live)
    content."""
import cv2
import sys
import json

cap = cv2.VideoCapture(sys.argv[1])
_, prev = cap.read()
_, curr = cap.read()
frame_count = 1
motion_count = 0
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Sample every 30th frame
while curr is not None:
    diff = cv2.absdiff(cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY),
                      cv2.cvtColor(curr, cv2.COLOR_BGR2GRAY))
    motion = cv2.countNonZero(diff) > 500 # threshold
    if motion:
        motion_count += 1
    prev = curr
    for _ in range(30):
        ret, curr = cap.read()
        if not ret:
            curr = None
            break
    frame_count += 30

result = {
    "live": motion_count > 0,
    "motion_frames": motion_count,
    "total_frames_sampled": frame_count,
    "pass": motion_count > 0
}
print(json.dumps(result))
cap.release()
```

Bash pipeline:

```
python3 motion_check.py recording.mp4 | jq '.pass'
# true = video has motion (live content)
```

2.3.2 Perceptual Frame Hashing (Not-stale Check)

```
#!/usr/bin/env python3
"""not_stale.py – Verify consecutive frames are not duplicates
(stale decoder)."""
import cv2, sys, json
from skimage.metrics import structural_similarity as ssim

cap = cv2.VideoCapture(sys.argv[1])
_, f1 = cap.read()
_, f2 = cap.read()

g1 = cv2.cvtColor(f1, cv2.COLOR_BGR2GRAY)
g2 = cv2.cvtColor(f2, cv2.COLOR_BGR2GRAY)

score = ssim(g1, g2)
print(json.dumps({
    "ssim_score": float(score),
    "is_live": score < 0.95,    # SSIM < 0.95 means frame is
                              advancing
    "pass": score < 0.95
}))
```

2.3.3 OCR on Video Frames (OpenCV + Tesseract)

```
#!/usr/bin/env python3
"""ocr_video.py – Extract text from a video at specific
timestamps."""
import cv2, sys, json
import pytesseract
from PIL import Image

cap = cv2.VideoCapture(sys.argv[1])
timestamps = [float(t) for t in sys.argv[2:]] # seconds

results = []
for ts in timestamps:
    cap.set(cv2.CAP_PROP_POS_FRAMES,
            int(cap.get(cv2.CAP_PROP_FPS) * ts))
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY +
                              cv2.THRESH_OTSU)
    text = pytesseract.image_to_string(thresh, config='--psm 6')
    results.append({"ts": ts, "text": text.strip()})
```

```
print(json.dumps(results))
cap.release()
```

Local / offline: YES.

License: Apache 2.0.

3. Audio Analysis Tools

3.1 ffmpeg Audio Analysis

Purpose: Audio stream extraction, format conversion, silence detection, channel analysis.

Installation: Via ffmpeg (see §2.1).

3.1.1 Audio Stream Detection

```
# List audio streams
ffprobe -v quiet -print_format json -show_entries
        stream=index,codec_name,channels,sample_rate input.mp4 \
    | jq '.streams[] | select(.codec_type == "audio")'
```

3.1.2 Audio Extraction

```
# Extract audio to WAV (16kHz mono, for speech recognition)
ffmpeg -i input.mp4 -vn -acodec pcm_s16le -ar 16000 -ac 1
        output.wav
```

```
# Extract with original quality
ffmpeg -i input.mp4 -vn -acodec copy output.aac
```

3.1.3 Silence Detection (silencedetect filter)

```
# Detect silence in audio
ffmpeg -i input.wav -af "silencedetect=n=-30dB:d=0.5" -f null -
        2>&1 \
    | grep -oP '(silence_start|silence_end|silence_duration):
        \K[0-9.]+'
```

Parameters: - n: noise threshold in dB (default -30dB). Lower = more sensitive. - d: minimum silence duration in seconds (default 0.5).

3.1.4 RMS / Loudness Analysis

```
# Per-frame RMS
ffmpeg -i input.wav -af "volumedetect" -f null - 2>&1
```

```
# EBU R128 loudness (LUFS)
ffmpeg -i input.wav -af "ebur128" -f null - 2>&1
```

```
# Extract RMS values as CSV-like lines
ffmpeg -i input.wav -af "astats=metadata=1" -f null - 2>&1 \
  | grep "RMS_level" | head -20
```

3.1.5 Audio Quality Checks in Bash

```
#!/bin/bash
# verify_audio.sh - Check audio presence and basic quality

INPUT="$1"

# Check if audio stream exists
STATUS=$(ffprobe -v error -select_streams a:0 \
  -show_entries stream=codec_name \
  -of default=noprint_wrappers=1:nokey=1 "$INPUT")

if [ -z "$STATUS" ]; then
  echo "FAIL: No audio stream found"
  exit 1
fi

# Check channels
CHANNELS=$(ffprobe -v error -select_streams a:0 \
  -show_entries stream=channels \
  -of default=noprint_wrappers=1:nokey=1 "$INPUT")
echo "Channels: $CHANNELS"

# Check sample rate
RATE=$(ffprobe -v error -select_streams a:0 \
  -show_entries stream=sample_rate \
  -of default=noprint_wrappers=1:nokey=1 "$INPUT")
echo "Sample rate: $RATE Hz"

# Check for silence (if silent, fail)
if ffmpeg -i "$INPUT" -af "silencedetect=n=-50dB:d=2" -f null -
  2>&1 \
  | grep -q "silence_end: inf"; then
  echo "FAIL: Audio is entirely silent"
  exit 1
fi

echo "PASS: Audio stream present and non-silent"
```

3.2 OpenAI Whisper (Local)

Purpose: State-of-the-art speech-to-text transcription, fully local. No API key needed.

How it works: Encoder-decoder Transformer trained on 680k hours of multilingual data. Predicts text directly from 30-second audio windows.

Installation:

```
pip install openai-whisper
# or faster backends:
pip install faster-whisper      # CTranslate2 – 4x faster
pip install whisperly           # auto-selects fastest backend
```

Model selection (2025-2026 recommended):

	Model	Size	RAM	Speed	Best for
tiny	75 MB	~1 GB	Fastest	Real-time, quick check	
base	145 MB	~2 GB	Fast	General use	
small	465 MB	~4 GB	Medium	Balanced	
medium	1.5 GB	~8 GB	Slow	Good accuracy	
large-v3-turbo	~1.5 GB	~6 GB	Medium-fast	Best 2025-2026 choice	
large	2.9 GB	~10 GB	Slow	Maximum accuracy	

Bash pipeline — transcribe video to text:

```
#!/bin/bash
# transcribe.sh – Extract audio and transcribe with Whisper

INPUT="$1"
MODEL="${2:-large-v3-turbo}"
OUTDIR="${3:-./transcript}"

mkdir -p "$OUTDIR"

# Step 1: Extract audio at 16kHz mono
ffmpeg -i "$INPUT" -vn -acodec pcm_s16le -ar 16000 -ac 1 "$OUTDIR/
audio.wav" -y

# Step 2: Transcribe (fully offline)
whisper "$OUTDIR/audio.wav" \
  --model "$MODEL" \
  --output_dir "$OUTDIR" \
  --output_format srt txt \
  --language en

echo "Transcript: $OUTDIR/audio.txt"
echo "Subtitles: $OUTDIR/audio.srt"
```

Pattern matching in transcripts for verification:

```
# Check if transcript contains expected text
if grep -qi "expected output" transcript/audio.txt; then
    echo "PASS: Expected text found in audio"
else
    echo "FAIL: Expected text NOT found in audio"
fi
```

Device auto-detection for pipeline:

```
DEVICE="cpu"
if command -v nvidia-smi &> /dev/null; then DEVICE="cuda"
elif [ "$(uname)" = "Darwin" ] && [ "$(uname -m)" = "arm64" ];
then DEVICE="mps"
```

```
fi
whisper audio.wav --model large-v3-turbo --device "$DEVICE" --
    output_format txt
```

Local / offline: YES after initial model download. No API key.

License: MIT.

Use for §11.4.159: Transcribe audio tracks from feature recordings to verify the correct audio output, captions, or spoken content is present.

3.3 librosa

Purpose: Python library for music and audio analysis. Feature extraction (MFCC, spectral), beat tracking, onset detection.

Installation:

```
pip install librosa numpy
```

Key capabilities for media validation:

```
import librosa
import json

# Load audio
y, sr = librosa.load('audio.wav', sr=16000)

# RMS energy (loudness) – detect silent/dead channels
rms = librosa.feature.rms(y=y)
mean_rms = float(rms.mean())
print(json.dumps({"mean_rms": mean_rms, "has_audio": mean_rms >
    0.01}))

# Spectral centroid (brightness)
centroid = librosa.feature.spectral_centroid(y=y, sr=sr)

# Zero-crossing rate (noise detection)
zcr = librosa.feature.zero_crossing_rate(y)

# Detect if file is silent
if mean_rms < 0.001:
    print("FAIL: Audio is silent")
else:
    print("PASS: Audio has content")
```

Bash integration:

```
python3 -c "
import librosa, json
y, sr = librosa.load('audio.wav', sr=16000)
rms = librosa.feature.rms(y=y)
```

```
print(json.dumps({'mean_rms': float(rms.mean()), 'channels':  
                  y.ndim}))  
"
```

Local / offline: YES.

License: ISC.

4. Asciiinema Cast Analysis

4.1 agg (asciiinema GIF generator)

Purpose: Convert .cast (asciiinema terminal recording) files to GIF for visual review. Successor to asciicast2gif.

Installation:

```
# Via Rust (cargo)  
cargo install --git https://github.com/asciiinema/agg  
  
# Via Homebrew  
brew install agg
```

Usage:

```
# Convert cast to GIF  
agg recording.cast recording.gif  
  
# With theme and speed options  
agg --theme monokai --speed 2.0 --font-size 14 recording.cast  
    recording.gif  
  
# Convert to MP4 (via ffmpeg)  
agg recording.cast recording.gif  
ffmpeg -i recording.gif -c:v libx264 -pix_fmt yuv420p -movflags  
    +faststart recording.mp4
```

Bash pipeline for automated verification:

```
#!/bin/bash  
# cast_to_mp4.sh - Convert asciicast to viewable MP4 then run OCR  
  
INPUT="$1"  
BASE="${INPUT%.cast}"  
  
# Step 1: Convert cast -> GIF -> MP4  
agg "$INPUT" "${BASE}.gif"  
ffmpeg -y -i "${BASE}.gif" -c:v libx264 -pix_fmt yuv420p "$  
    {BASE}.mp4"  
  
# Step 2: Extract frames from MP4 for OCR  
mkdir -p "${BASE}_frames"
```



```
ffmpeg -i "${BASE}.mp4" -vf "fps=1" "${BASE}_frames/
    frame_%04d.png"

# Step 3: OCR each frame
for f in "${BASE}_frames"/*.png; do
    tesseract "$f" "${f%.png}" -l eng --psm 6 2>/dev/null
done

cat "${BASE}_frames"/*.txt > "${BASE}_transcript.txt"
echo "Transcript: ${BASE}_transcript.txt"
```

Debugging tip: `agg --debug` prints timing, render info, and error diagnostics.

Local / offline: YES.

License: GPL 3.0.

4.2 asciinema-rewind (community)

Purpose: Extract metadata and timing information from `.cast` files.

Installation:

```
pip install asciinema-rewind
```

Usage:

```
# Extract timing and event counts
asciinema-rewind recording.cast
```

Local / offline: YES.

4.3 Direct `.cast` JSON Parsing

Purpose: Ascinema `.cast` files are JSONL — each line is a timestamped event. Parse directly without conversion.

Format of a `.cast` file (v2):

```
{"version": 2, "width": 80, "height": 24, "timestamp": 1712345678}
[0.1, "o", "$ "]
[0.2, "o", "ls -la"]
[0.05, "o", "\r\n"]
[0.3, "o", "total 42\r\n"]
[1.0, "o", "PASS: All tests completed successfully\r\n"]
```

Bash — grep for expected output in cast:

```
#!/bin/bash
# verify_cast.sh - Check if asciicast contains expected output
# patterns
```

```

INPUT="$1"
PATTERN="${2:-PASS}"

# Extract text output (lines with type "o") and grep for pattern
if grep -E '^\[' "$INPUT" | grep -E '"o",' | grep -qi "$PATTERN";
then
    echo "PASS: Pattern '$PATTERN' found in cast"
    exit 0
else
    echo "FAIL: Pattern '$PATTERN' NOT found in cast"
    exit 1
fi

```

Python — structured cast verification:

```

#!/usr/bin/env python3
"""verify_cast.py - Structured verification of asciicast
output."""
import sys, json

cast_file = sys.argv[1]
expected_patterns = sys.argv[2:]

output_text = ""
with open(cast_file) as f:
    for line in f:
        line = line.strip()
        if line.startswith('['):
            try:
                event = json.loads(line)
                if len(event) >= 3 and event[1] == 'o':
                    output_text += event[2]
            except json.JSONDecodeError:
                pass

found_all = True
for pattern in expected_patterns:
    if pattern.lower() in output_text.lower():
        print(f"PASS: '{pattern}' FOUND")
    else:
        print(f"FAIL: '{pattern}' NOT FOUND")
        found_all = False

sys.exit(0 if found_all else 1)

```

Bash usage:

```
python3 verify_cast.py session.cast "PASS" "completed
successfully"
```

Local / offline: YES. Zero dependencies for JSON parsing.

5. Automated Content Verification Frameworks

5.1 HelixQA (Existing Submodule)

Purpose: The project's own automated QA framework. Drives end-to-end validation with HelixQA Challenge banks.

Repository: HelixDevelopment/HelixQA (git submodule).

How it works: Challenge-driven QA sessions. Each challenge defines an expected behaviour, drives the system under test, captures evidence, and scores PASS/FAIL. Uses subagent-driven execution per §11.4.70.

Verification via docs/qa/<run-id>/ (§11.4.83): - Full bidirectional transcript - Evidence artefacts (recordings, screenshots, logs) - Verdict with evidence path

Pipeline integration:

```
# Run a HelixQA session from scripts/testing/  
bash scripts/testing/run_qa_session.sh --bank atmosphere --  
challenge CME-MEDIA-001
```

Key features for media validation: - Subagent-driven parallel execution - Captured evidence output under qa-results/<run-id>/ - Challenge bank under tools/helixqa/banks/ - ab_pass_with_evidence / ab_skip_with_reason helpers per §11.4.69 - ab_run_n_times for §11.4.50 deterministic consistency

Local / offline: YES.

License: Proprietary (owned submodule).

5.2 Playwright + axe-core

Purpose: Browser automation framework with accessibility testing. Useful for web/UI-based media validation.

Installation:

```
npm init playwright  
npm install @axe-core/playwright --save-dev
```

Video recording built-in:

```
// playwright.config.js  
module.exports = defineConfig({  
  use: {  
    video: 'on',           // Record video for all tests  
    screenshot: 'on',      // Capture screenshots  
  }  
});
```

Accessibility audit:

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test('page has no accessibility violations', async ({ page }) => {
  await page.goto('https://example.com');
  const results = await new AxeBuilder({ page }).analyze();
  expect(results.violations).toEqual([]);
});
```

Bash CI pipeline:

```
npx playwright test --headless --reporter html
# Videos in test-results/ directory
```

Pros: Rich assertions, auto-waiting, video recording, cross-browser, extensive ecosystem. **Cons:** Only for web/browser-based UI. Not for terminal or device-native media.

Local / offline: YES.

License: Apache 2.0.

5.3 Cypress

Purpose: End-to-end testing framework with built-in video recording and time-travel debugging.

Installation:

```
npm install cypress --save-dev
```

Video recording configuration:

```
// cypress.config.js
module.exports = defineConfig({
  e2e: {
    video: true,
    videoCompression: 32,
    videoUploadOnPasses: false, // upload only on failure
    videosFolder: 'cypress/videos',
  }
});
```

CI pipeline:

```
npx cypress run --headless --browser chrome
```

Pros: Built-in video recording on every test run, time-travel debugging, extensive plugin ecosystem. **Cons:** Only for web/browser-based UI. Heavier than Playwright.

Local / offline: YES.

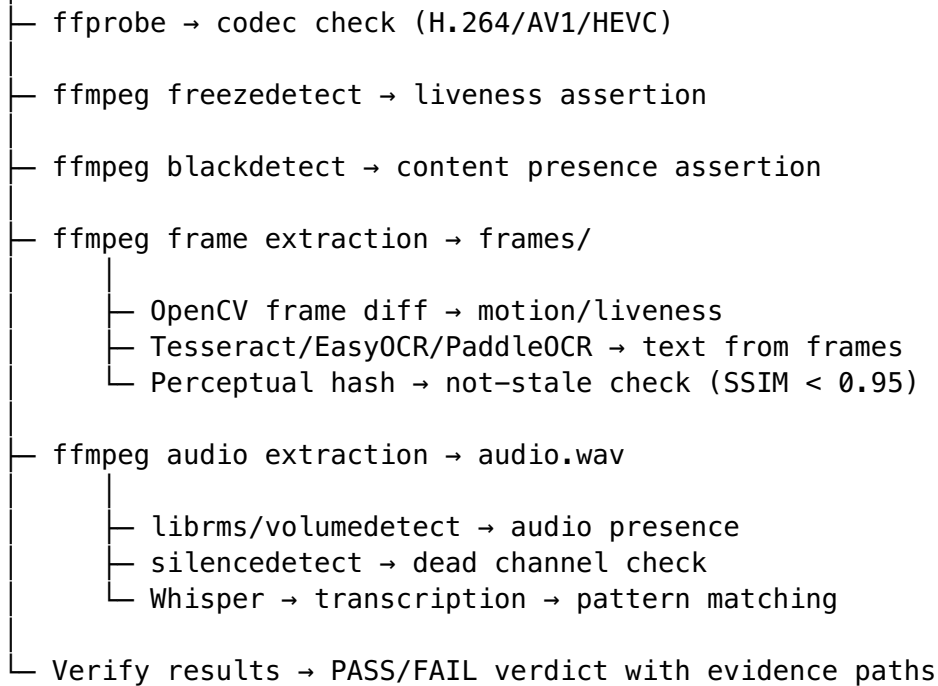
License: MIT.

5.4 Custom Python Pipeline

Purpose: Fully custom bash/Python pipeline combining ffmpeg + OCR + Whisper + OpenCV. Most flexible for media validation.

Architecture:

Input video/recording



Template pipeline script:

```
#!/bin/bash
# media_validate.sh - Comprehensive media recording validation

INPUT="$1"
TOOL="${2:-standard}"

echo "=== Media Validation Report ==="
echo "Input: $INPUT"
echo "Tool: $TOOL"
echo "===== "

# --- Phase 1: Codec check ---
CODEC=$(ffmpeg -v error -select_streams v:0 \
    -show_entries stream=codec_name \
    -of default=noprint_wrappers=1:nokey=1 "$INPUT" 2>/dev/null)
echo "Video codec: ${CODEC:-N/A}"

if [ "$CODEC" = "h264" ]; then
    echo " [PASS] Codec is H.264"
else
    echo " [WARN] Expected H.264, got $CODEC"
```

```

fi

# --- Phase 2: Freeze detection (liveness) ---
FREEZE=$(ffmpeg -i "$INPUT" -vf "freezedetect=n=-60dB:d=2" \
    -an -f null - 2>&1 | grep -c "freeze_duration")
if [ "$FREEZE" -eq 0 ]; then
    echo " [PASS] No freeze detected (video is live)"
else
    echo " [FAIL] $FREEZE freeze event(s) detected"
fi

# --- Phase 3: Audio presence ---
AUDIO=$(ffprobe -v error -select_streams a:0 \
    -show_entries stream=codec_name \
    -of default=noprint_wrappers=1:nokey=1 "$INPUT" 2>/dev/null)

if [ -n "$AUDIO" ]; then
    echo "Audio codec: $AUDIO"
    # Check for silence
    ffmpeg -i "$INPUT" -af "volumedetect" -f null - 2>&1 \
        | grep "mean_volume" | head -1
    echo " [PASS] Audio stream present"
else
    echo " [WARN] No audio stream"
fi

# --- Phase 4: Frame extraction + OCR ---
mkdir -p /tmp/media_val_frames
ffmpeg -y -i "$INPUT" -vf "fps=1/5" /tmp/media_val_frames/
    frame_%04d.png 2>/dev/null

echo "--- Phase 4: OCR verification ---"
for f in /tmp/media_val_frames/*.png; do
    text=$(tesseract "$f" stdout -l eng --psm 6 2>/dev/null |
        head -5)
    if [ -n "$text" ]; then
        echo " [OCR] $(basename "$f"): ${text:0:80}"
    fi
done
rm -rf /tmp/media_val_frames

echo "=== End of Media Validation Report ==="

```

Python orchestrator equivalent:

```

#!/usr/bin/env python3
"""media_validate.py - Orchestrated media validation pipeline."""
import subprocess, json, sys
from pathlib import Path

def check_codec(video_path: str) -> dict:
    result = subprocess.run([
        'ffprobe', '-v', 'error', '-select_streams', 'v:0',

```

```

        '-show_entries', 'stream=codec_name',
        '-of', 'default=noprint_wrappers=1:nokey=1', video_path
    ], capture_output=True, text=True)
    codec = result.stdout.strip()
    return {'codec': codec, 'is_h264': codec == 'h264',
            'pass': codec == 'h264'}

def check_freezes(video_path: str) -> dict:
    result = subprocess.run([
        'ffmpeg', '-i', video_path,
        '-vf', 'freezedetect=n=-60dB:d=2',
        '-an', '-f', 'null', '-'
    ], capture_output=True, text=True)
    freeze_count = result.stderr.count('freeze_duration')
    return {'freeze_count': freeze_count,
            'pass': freeze_count == 0}

def check_audio(video_path: str) -> dict:
    result = subprocess.run([
        'ffprobe', '-v', 'error', '-select_streams', 'a:0',
        '-show_entries', 'stream=codec_name,channels,sample_rate',
        '-of', 'json', video_path
    ], capture_output=True, text=True)
    streams = json.loads(result.stdout).get('streams', [])
    return {'has_audio': len(streams) > 0,
            'details': streams[0] if streams else None,
            'pass': len(streams) > 0}

if __name__ == '__main__':
    path = sys.argv[1]
    results = {
        'codec': check_codec(path),
        'freeze': check_freezes(path),
        'audio': check_audio(path),
        'overall_pass': all([
            check_codec(path)['pass'],
            check_freezes(path)['pass'],
            check_audio(path)['pass'],
        ])
    }
    print(json.dumps(results, indent=2))
    sys.exit(0 if results['overall_pass'] else 1)

```

Local / offline: YES (every component runs locally).

6. Comparison Matrix

OCR / Vision Tools

Feature	Tesseract	EasyOCR	PaddleOCR v4	Surya
Accuracy (overall)	~88-92%	~89%	~93.5%	~94.8%
Table recognition	No	No	Yes (built-in)	Yes (built-in)
Layout analysis	No	No	Yes (PP-Structure)	Yes (comprehensive)
Reading order	No	No	No	Yes
Handwriting	Poor (~60%)	Poor (~76%)	Fair (~82%)	Good (~85%)
CJK support	Good	Good	Best	Good
Speed (CPU)	Fastest	Slow	Medium	Slow (needs GPU)
GPU required	No	Optional	Optional	Recommended
VRAM usage	N/A	~3.1 GB	~2.8 GB	~4.2 GB
Installation	apt/brew simple	pip simple	pip (PaddlePaddle)	pip (models 1.3 GB)
Local / offline	YES	YES	YES	YES
License	Apache 2.0	Apache 2.0	Apache 2.0	GPL 3.0
Best for	Quick/lightweight	Prototyping	Production/CJK	Max accuracy

Video Analysis Tools

Feature	ffmpeg/ffprobe	PySceneDetect	OpenCV custom
Codec detection	YES (built-in)	No	No
Freeze detection	YES (freezedetect)	No	YES (frame diff)
Black frame detection	YES (blackdetect)	No	YES (cv2)
Scene detection	YES (select filter)	YES (primary function)	YES (cv2 diff)
Frame extraction	YES	YES (save-images)	YES (cv2)
Motion detection	No	No	YES (cv2)
Perceptual hash	No	No	YES (imagehash)

Feature	ffmpeg/ffprobe	PySceneDetect	OpenCV custom
OCR on frames	No	No	YES (+tesseract)
SSIM/PSNR metrics	YES	No	YES (skimage)
Local / offline	YES	YES	YES
License	LGPL/GPL	BSD 3-Clause	Apache 2.0
Best for	Foundation swiss-army-knife	Scene detection & splitting	Custom frame-level analysis

Audio Analysis Tools

Feature	ffmpeg (built-in)	Whisper	librosa
Audio extraction	YES	Needs ffmpeg	Needs ffmpeg
Silence detection	YES (silencedetect)	No	No
Loudness/RMS	YES (volumedetect)	No	YES
EBU R128 LUFS	YES (ebur128)	No	No
Speech transcription	No	YES (core function)	No
MFCC/spectral features	No	No	YES
Format conversion	YES	Needs ffmpeg	Needs ffmpeg
Local / offline	YES	YES	YES
License	LGPL/GPL	MIT	ISC
Best for	Audio presence + quality	Speech → text	Feature extraction

Asciicast Tools

Feature	agg	asciinema-rewind	Direct JSON parse
Cast → GIF/MP4	YES	No	No
Timing extraction	No	YES	YES
Text extraction	No	No	YES (type “o” events)
Pattern matching	No	No	YES (grep/jq)
Local / offline	YES	YES	YES (zero deps)
License	GPL 3.0	MIT	N/A

Full-Stack Frameworks

Feature	HelixQA	Playwright	Cypress	Custom Python
Media validation	YES (extensible)	Web-only	Web-only	YES
Challenge banks	YES (built-in)	No	No	No
On-device testing	YES	No	No	YES
Subagent support	YES (§11.4.70)	No	No	Via bash
Video recording	Via §11.4.128	Built-in	Built-in	Via ffmpeg
OCR integration	Via tools	Via axe-core	Screenshot-based	YES (all)
Audio validation	Via tools	No	No	YES
License	Proprietary	Apache 2.0	MIT	N/A
Best for	Full-system QA	Web UI testing	Web E2E	Maximum flexibility

7. Recommended Pipeline Architecture

Based on the research, the recommended media validation pipeline for Helix OTA §11.4.163 composes:

Media Recording

```
├─ [LEVEL 1: FORMAT VALIDATION]
│   ├── ffprobe → codec == h264? (PASS/FAIL)
│   ├── ffprobe → audio stream present? (PASS/WARN)
│   └── ffprobe → resolution, bitrate, framerate
├─ [LEVEL 2: LIVENESS DETECTION]
│   ├── ffmpeg freezedetect → freeze_count == 0? (PASS/FAIL)
│   ├── OpenCV frame diff → SSIM < 0.95 between consecutive frames
│   └── ffmpeg blackdetect → no extended black segments
├─ [LEVEL 3: CONTENT VERIFICATION]
│   ├── For terminal recordings (.cast):
│   │   ├── JSON parse → grep for expected PASS/PATTERNS
│   │   └── agg → GIF → ffmpeg → MP4 → tesseract → text grep
│   └── For GUI/display recordings (.mp4):
│       ├── ffmpeg frame extraction → tesseract/EasyOCR
│       │   └── → grep for expected UI labels/status text
│       └── OpenCV ROI crop → OCR on specific regions
```

- └─ For audio content:
 - Whisper → transcription → grep for expected phrases
 - ffmpeg volumedetect → RMS > threshold
- └─ [LEVEL 4: SUBTITLE/OVERLAY VALIDATION per §11.4.137]
 - Surya layout analysis → subtitle region detection
 - ROI OCR at subtitle safe zone (CEA-708 9-anchor)
 - Content-class → CHROME vs DIALOGUE classification
- └─ [LEVEL 5: REPORT + EVIDENCE]
 - JSON verdict → docs/qa/<run-id>/evidence.json
 - Captured evidence paths per §11.4.69
 - Overall PASS/FAIL for HelixQA challenge scoring

Tool Selection by Use Case

Use Case	Recommended Tool Chain
Quick format + codec check	ffprobe
Terminal feature recording validation	JSON parse .cast + agg → tesseract
GUI/display live content	ffmpeg freezedetect + OpenCV frame diff + tesseract
Audio content verification	ffmpeg extraction + Whisper transcription + grep
Document/overlay OCR	Surya for layout + tesseract for content
High-volume batch OCR	PaddleOCR
Simple screen text capture	tesseract + ImageMagick preprocessing
Full HelixQA challenge session	HelixQA suite + custom validator script

Implementation Priority (recommended)

1. **Phase 1** — ffprobe format checks + ffmpeg freezedetect (quick wins, zero Python deps)
2. **Phase 2** — tesseract OCR on extracted frames + .cast JSON parsing (bash-only)
3. **Phase 3** — OpenCV frame diff + motion detection + perceptual hashing
4. **Phase 4** — Whisper transcription + pattern matching
5. **Phase 5** — Surya/PaddleOCR for higher-accuracy OCR + layout analysis

Sources

- [Tesseract OCR — official docs](#)
- [EasyOCR — GitHub](#)
- [PaddleOCR — GitHub](#)
- [Surya OCR — GitHub](#)

- [GigaGPU OCR Model Rankings \(2026\)](#)
- [ffmpeg official documentation](#)
- [ffmpeg freezedetect filter docs](#)
- [PySceneDetect — GitHub](#)
- [OpenCV — official docs](#)
- [OpenAI Whisper — GitHub](#)
- [faster-whisper — GitHub](#)
- [librosa — official docs](#)
- [agg — asciinema GIF generator](#)
- [asciinema JSON format specification](#)
- [Playwright — official docs](#)
- [Cypress — official docs](#)
- [HelixQA — internal submodule](#)
- [whisply — batch Whisper CLI](#)
- [ffmpeg silence detection docs](#)